RAILWAY RESERVATION SYSTEM

Software Engineering Mini Project — Part 1 Deliverables

Team E15

Project members and individual responsibility

Deliverable 1 — Software Requirements Specification (SRS): Mohit Singh P, PES1UG24CS273.

Deliverable 2 — Test Plan: Atharv More, PES1UG24CS276.

Deliverable 3 — Software Architecture and Design Specification: Mohammed Rizwan shaikh, PES1UG24CS272.

Deliverable 4 — Test case update within Test Plan §5.2: Muhammad Affan Dumba, PES1UG24CS280.

Document scope: an academic prototype for searching trains, reserving seats, managing bookings, and administering train inventory. Payment and refund actions use a simulator; no real payment is processed. This assumption must be confirmed if a live gateway is required.

Document conventions: FR = functional requirement; NFR = non-functional requirement; SEC = security requirement; TC = test case. “Shall” statements are mandatory and verifiable. Times are measured at the service boundary under the test conditions stated below.

DELIVERABLE 1 — SOFTWARE REQUIREMENTS SPECIFICATION

Prepared by Mohit Singh (PES1UG24CS273). Structured using the requirement principles of ISO/IEC/IEEE 29148:2018.

1. Introduction

1.1 Purpose

This specification defines the observable behavior, interfaces, constraints, and security requirements of the Railway Reservation System (RRS) prototype. It is the baseline for design and test traceability.

1.2 Scope

Passengers register, sign in, search trains, check seat availability, create a reservation after simulated payment, view bookings, and cancel eligible bookings. Administrators maintain trains, schedules, seat inventory, and review audit records. The prototype excludes actual railway network integration, real money movement, dynamic pricing, and waitlist allocation.

1.3 Definitions

PNR: unique booking reference. Seat hold: a temporary exclusive claim on one seat during booking. Confirmed booking: a booking with a seat and successful simulated payment. Administrator: authorized user permitted to manage schedules and inventory.

1.4 References

Assignment: “SE Mini Project Deliverables Part-1” supplied in the project folder. Requirements guidance: ISO/IEC/IEEE 29148:2018, https://standards.ieee.org/ieee/29148/6937/.

2. Overall description

2.1 Product perspective and users

RRS is a browser-based client with a REST API and a relational database. Passenger and administrator are the two user roles. A payment simulator returns success or failure deterministically for test purposes.

2.2 Operating environment

A current desktop or mobile browser connects over HTTPS to the application. The API and database run on a server. All timestamps are stored in UTC and shown with a timezone label.

2.3 Assumptions and constraints

A train has a train number, origin, destination, departure and arrival times, travel date, fare, and finite seat inventory. A passenger books one seat per transaction. A cancellation is allowed until the train departure time. Test accounts and simulated payment responses are available in the test environment.

3. External interface requirements

User interface: passenger screens for registration, sign-in, search, availability, checkout, bookings, and cancellation; administrator screens for train management and audit review. Forms shall identify required fields and show actionable validation errors.

Software interface: JSON over HTTPS. The payment simulator exposes an authorize operation and a refund operation. The database stores users, trains, trips, seats, bookings, payment events, and audit events.

4. Specific requirements

4.1 Functional requirements

FR-01 Registration. The system shall create a passenger account when a unique email, a name, and a password of at least 12 characters are submitted; it shall reject duplicate email addresses without creating another account.

FR-02 Authentication. The system shall issue an authenticated session only after valid credentials are supplied and shall reject an invalid password with a generic error message.

FR-03 Train search. Given an origin, destination, and travel date, the system shall return only trips that match all three fields and depart on or after the requested date, ordered by departure time.

FR-04 Availability. For each returned trip, the system shall show the number of unreserved seats; the displayed count shall never be below zero or above the trip capacity.

FR-05 Booking. When an authenticated passenger selects an available trip and the payment simulator returns success, the system shall allocate exactly one available seat and create exactly one confirmed booking with a unique PNR. If payment fails, it shall release the seat hold and create no confirmed booking.

FR-06 Booking history. The system shall show a passenger only that passenger's bookings, including PNR, trip details, seat, fare, and status.

FR-07 Cancellation. For a confirmed booking owned by the signed-in passenger and requested before departure, the system shall mark it cancelled, release its seat exactly once, and record a simulated refund reference. A repeat cancellation shall not release a second seat.

FR-08 Administration. An administrator shall create, edit, or disable a train trip and its seat capacity; a non-administrator shall receive HTTP 403 for those operations.

FR-09 Audit. The system shall record successful and failed sign-ins, booking confirmations and cancellations, and administrator trip changes with timestamp, actor identifier, action, and outcome.

4.2 Non-functional requirements

NFR-01 Search performance. With 10,000 stored trips and 50 concurrent users, the API shall return 95% of valid search requests within 2 seconds, measured over 200 requests in the test environment.

NFR-02 Booking performance. With 50 concurrent users, 95% of booking requests shall complete within 3 seconds, excluding an injected payment-simulator delay longer than 1 second, measured over 100 requests.

NFR-03 Concurrency. When 20 concurrent requests attempt to reserve the final seat of a trip, at most one shall be confirmed and the available seat count shall remain zero.

NFR-04 Availability and recovery. After an application process restart, committed bookings shall remain available and the service shall respond to a health request within 60 seconds.

NFR-05 Accessibility. The passenger and administrator interfaces shall support keyboard-only operation for the required workflows and expose programmatic labels for every input and button.

NFR-06 Compatibility. The required workflows shall work in the latest stable versions of Chrome, Edge, and Firefox at viewport widths of 375 pixels and 1280 pixels without horizontal scrolling of primary forms.

4.3 Security objectives and requirements

Objective SO-01: prevent unauthorized access to passenger records and administrator functions. Objective SO-02: protect credentials and booking data in transit and at rest. Objective SO-03: preserve an auditable history of security-sensitive actions.

SEC-01 Access control. Every booking read, cancellation, and administrator API request shall validate the authenticated user and role on the server; requests for another passenger's booking shall return HTTP 403 or 404 without booking details.

SEC-02 Credential protection. Passwords shall be stored only as salted adaptive hashes; plaintext passwords shall not appear in the database, logs, or API responses.

SEC-03 Transport and session. All authenticated API traffic shall use HTTPS; session tokens shall expire after 30 minutes of inactivity and be invalid after sign-out.

SEC-04 Abuse and audit. After five failed sign-in attempts for one account within 15 minutes, additional attempts shall be rejected for 15 minutes, and each rejection shall be logged without the submitted password.

5. UML use case diagram and use case notes

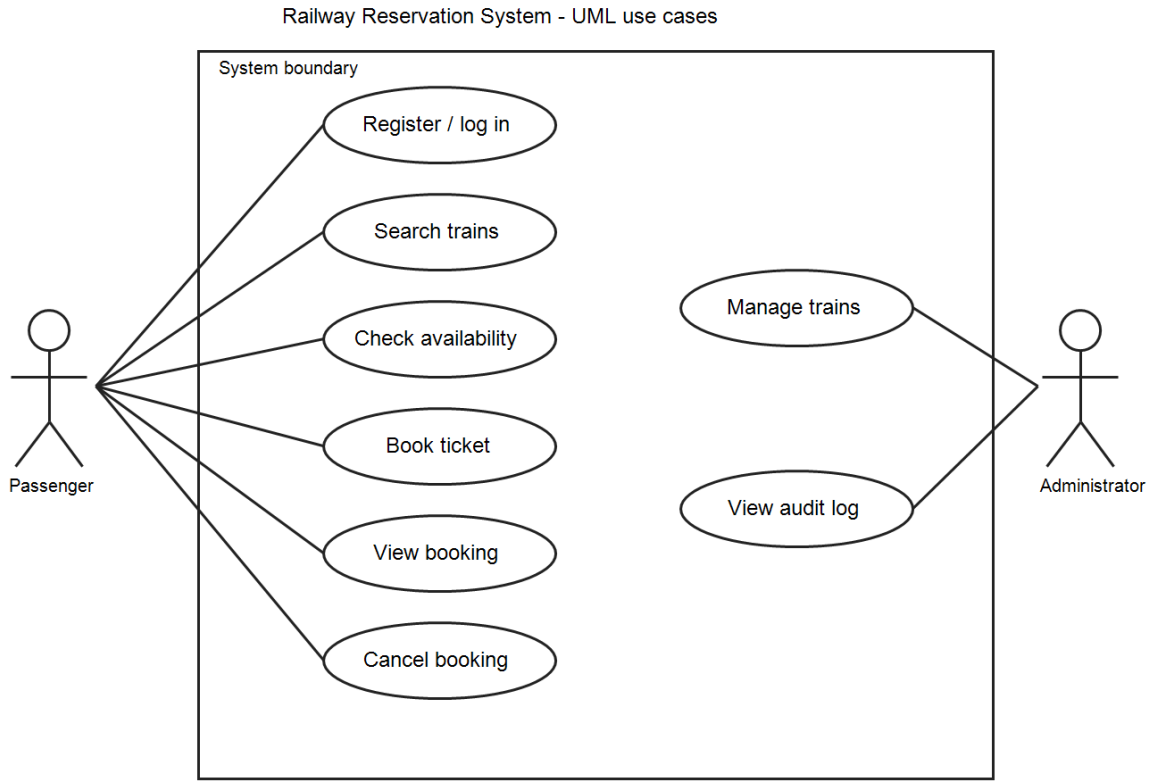


Figure 1. Passenger and administrator use cases inside the RRS boundary.

UC-01 Book ticket: the passenger signs in, searches a trip, checks availability, submits booking, receives simulated payment success, and receives a PNR. Failure alternatives are sold-out inventory and payment failure; neither may create a confirmed booking.

UC-02 Cancel booking: the passenger chooses a confirmed booking before departure; the system validates ownership, cancels once, frees the seat, and records a simulated refund reference. Repeated or late requests are rejected without changing inventory.

6. Acceptance basis

The system is acceptable for this Part 1 specification when each FR, NFR, and SEC requirement is linked to a planned test, the three UML diagram types are present, and the core booking and cancellation invariants are verified.

DELIVERABLE 2 — TEST PLAN

Prepared by Atharv More (PES1UG24CS276). Organized as an IEEE-style test plan aligned with ISO/IEC/IEEE 29119-3:2021. §5.2 is the separate Deliverable 4 contribution by Muhammad Affan Dumba.

1. Introduction

Purpose: define scope, strategy, resources, entry and exit criteria, security validation, and traceability for RRS system testing. Test items are the browser UI, REST API, database behavior, and payment simulator integration. The SRS in this document is the requirement baseline.

2. Test items and scope

In scope: registration, authentication, search, availability, booking, history, cancellation, administration, audit, performance, concurrency, accessibility, compatibility, and security. Out of scope: real card payments, external railway data feeds, and production disaster recovery.

3. Test strategy and approach

3.1 Levels: unit tests cover validation and seat state transitions; API integration tests cover authorization, persistence, and simulator responses; system tests cover passenger and administrator workflows in a browser.

3.2 Techniques: positive and negative scenarios, boundary values for password length and seat count, role-based access checks, duplicate-submission checks, and concurrent final-seat requests. Automated scripts are preferred for repeatable API, load, and security checks; keyboard and cross-browser checks may be manual.

3.3 Test data: use synthetic passenger accounts P1 and P2, administrator A1, trips with zero, one, and multiple seats, and payment simulator success/failure modes. Reset the test database to a known seed before each independent case.

3.4 Defect handling: record case ID, environment, steps, expected and actual results, severity, and evidence. Re-run a failed case after the fix and run the relevant regression cases.

4. Test environment, responsibilities, and schedule

4.1 Environment: HTTPS test deployment; relational test database seeded with 10,000 trips for performance tests; current Chrome, Edge, and Firefox; 375-pixel and 1280-pixel browser viewports; a load tool capable of 50 concurrent virtual users.

4.2 Responsibilities: Atharv More owns this plan and execution coordination. Muhammad Affan Dumba owns §5.2 test case specifications and coverage updates. Mohit Singh reviews requirement mapping. Mohammed Rizwan sheik reviews architecture-related setup and failure behavior.

4.3 Sequence: prepare and seed environment; run smoke tests; execute functional and security cases; run performance, concurrency, accessibility, and compatibility checks; record defects and regression results; review exit criteria.

4.4 Entry criteria: SRS and design are baselined; test build is deployed; seed data and simulator are available. Exit criteria: all ten cases have recorded results; no open critical or high defects remain; every FR, NFR, and SEC ID is covered by a case or an explicit follow-up test.

5. Test design and validation

The ten cases in §5.2 are specifications, not claims of execution. Actual result, date, tester, build, and pass/fail fields should be completed during testing.

5.1 Security validation

SV-01 Authentication: attempt five incorrect passwords, verify lockout on the next attempt for 15 minutes, generic error text, and audit events without passwords. Covers FR-02, FR-09, SEC-02, SEC-04; executed in TC-02 and TC-09.

SV-02 Authorization: use P2 to request P1's booking and use P1 on administrator endpoints; verify HTTP 403 or 404 with no protected data. Covers FR-06, FR-08, SEC-01; executed in TC-06 and TC-08.

SV-03 Transport and session: inspect HTTPS on authenticated requests; sign out and replay the old token; test expiry after 30 minutes of inactivity. Covers SEC-03; executed in TC-09.

DELIVERABLE 4 — TEST CASE UPDATE WITHIN TEST PLAN §5.2

Prepared by Muhammad Affan Dumba (PES1UG24CS280). Ten cases cover functional, non-functional, and security requirements. For each case, record actual result, evidence, tester, date, and status during execution.

TC-01 — Registration and duplicate email

Preconditions: No account exists for user P1.

Steps: Submit a valid 12-character password and unique email; repeat with the same email.

Expected result: First submission creates one account; second is rejected and account count remains one.

Traceability: FR-01; SEC-02.

TC-02 — Sign-in and failed attempts

Preconditions: P1 exists and is unlocked.

Steps: Sign in with correct password; sign out; submit an incorrect password five times and attempt once more.

Expected result: Correct sign-in succeeds; invalid attempts reveal no account details; sixth attempt is blocked for 15 minutes and audited without password content.

Traceability: FR-02, FR-09; SEC-02, SEC-04.

TC-03 — Search and availability

Preconditions: At least three seeded trips include matching, nonmatching, and sold-out records.

Steps: Search by origin, destination, and date; inspect ordering and seat counts.

Expected result: Only matching trips appear in departure order; counts are within 0 and capacity.

Traceability: FR-03, FR-04.

TC-04 — Successful booking

Preconditions: P1 is signed in; selected trip has one free seat; simulator is set to success.

Steps: Submit one booking request and query the new booking.

Expected result: Exactly one confirmed booking and unique PNR exist; one seat is allocated; booking appears in P1 history.

Traceability: FR-05, FR-06; NFR-03.

TC-05 — Payment failure and repeated submission

Preconditions: P1 is signed in; trip has one free seat; simulator is set to failure.

Steps: Submit booking; then set simulator to success and send the same client request twice with one idempotency key.

Expected result: Failure creates no confirmed booking and releases hold; duplicate success submissions yield one booking and one seat allocation.

Traceability: FR-05; NFR-03.

TC-06 — Booking ownership and cancellation

Preconditions: P1 owns a confirmed future booking; P2 is signed in separately.

Steps: P2 reads and cancels P1 booking; P1 then cancels twice.

Expected result: P2 receives 403 or 404 without details; P1 first cancellation succeeds with refund reference; second makes no further seat change.

Traceability: FR-06, FR-07; SEC-01.

TC-07 — Administrator inventory and audit

Preconditions: A1 and P1 accounts exist.

Steps: As P1 attempt to create a trip; as A1 create, edit, and disable a trip; inspect audit events.

Expected result: P1 receives 403; A1 changes persist; events contain actor, time, action, and outcome.

Traceability: FR-08, FR-09; SEC-01.

TC-08 — Concurrent final seat

Preconditions: Trip has exactly one available seat; 20 distinct signed-in passengers are ready.

Steps: Send 20 booking requests simultaneously with simulator success.

Expected result: At most one booking is confirmed; remaining requests report unavailable; available seat count is zero, never negative.

Traceability: FR-04, FR-05; NFR-03.

TC-09 — Security, session, and restart

Preconditions: HTTPS test deployment and a confirmed booking exist.

Steps: Inspect transport; sign out and replay token; wait 30 minutes idle and reuse session; restart application; inspect audit and booking.

Expected result: Authenticated traffic uses HTTPS; invalid or expired tokens fail; committed booking persists and health responds within 60 seconds; audit omits passwords.

Traceability: FR-09; SEC-02, SEC-03; NFR-04.

TC-10 — Performance, accessibility, and compatibility

Preconditions: 10,000 trips seeded; 50 virtual users; three supported browsers available.

Steps: Run 200 searches and 100 bookings at 50 users; complete search and booking by keyboard at 375 and 1280 pixels in each browser.

Expected result: At least 95% of searches finish within 2 s and bookings within 3 s under the stated simulator condition; required controls have labels; workflows work without horizontal form scrolling.

Traceability: NFR-01, NFR-02, NFR-05, NFR-06.

6. Test completion and deliverables

Test evidence comprises case results, API responses or screenshots, performance summaries, accessibility notes, a defect log, and a final requirement coverage review. A failed case is not waived without a documented decision and reason.

7. Requirement traceability review

FR-01→TC-01; FR-02→TC-02; FR-03–04→TC-03 and TC-08; FR-05→TC-04, TC-05, TC-08; FR-06–07→TC-06; FR-08→TC-07; FR-09→TC-02, TC-07, TC-09.

NFR-01–02→TC-10; NFR-03→TC-04, TC-05, TC-08; NFR-04→TC-09; NFR-05–06→TC-10. SEC-01→TC-06, TC-07; SEC-02→TC-01, TC-02, TC-09; SEC-03→TC-09; SEC-04→TC-02.

DELIVERABLE 3 — SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Prepared by Mohammed Rizwan sheik (PES1UG24CS272). Organized as a software design description consistent with IEEE 1016:2009.

1. Introduction and design goals

The design realizes the SRS using a browser client, REST API, domain services, and relational database. The key invariants are one confirmed booking per successful idempotent request, no double allocation of a seat, and server-side enforcement of role and ownership.

2. Architecture

2.1 Architectural pattern

A layered client–server pattern separates presentation, API and authorization, domain services, and persistence. The browser does not access the database directly. The API coordinates transactions across inventory and booking operations. The payment simulator is an external-style adapter with a defined interface.

2.2 UML component diagram and descriptions

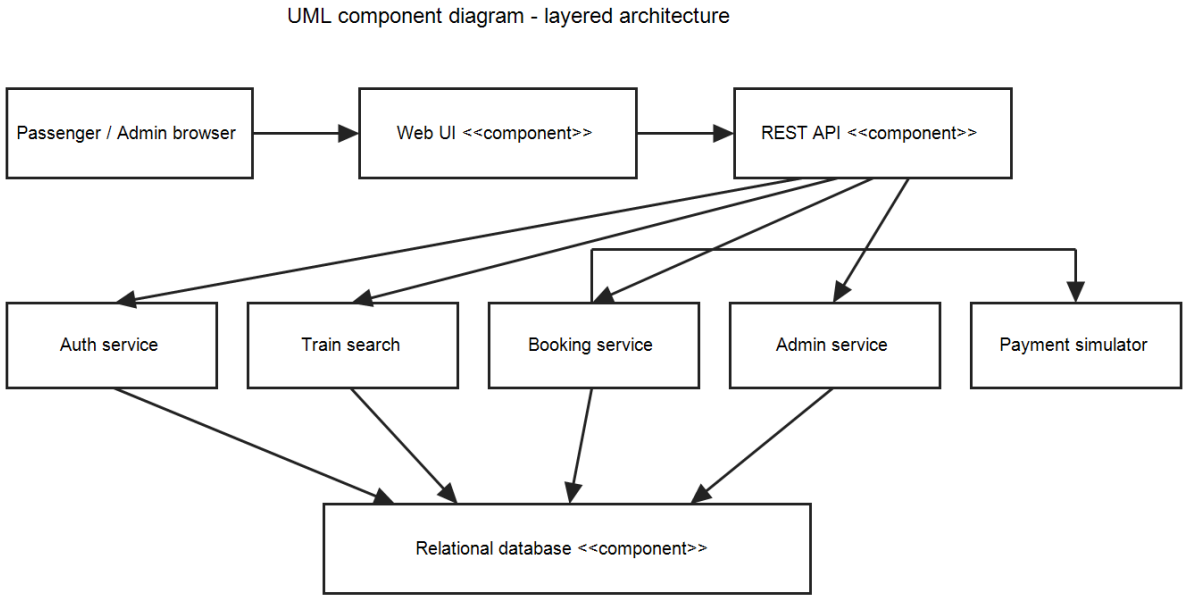


Figure 2. RRS components and their primary dependencies.

Web UI: gathers inputs, displays state and validation messages, and calls the REST API. REST API: authenticates requests, checks roles and ownership, validates payloads, and routes operations.

Auth service: verifies password hashes, sessions, and lockout policy. Train search: queries matching trips and calculated availability. Booking service: owns seat holds, confirmation, cancellation, idempotency, and PNR creation. Admin service: manages trip records and capacity. Payment simulator: returns test authorization and refund references. Relational database: enforces unique email, unique PNR, and exclusive seat allocation through constraints and transactions.

2.3 Data and deployment view

Main entities: User(id, email, password_hash, role); Trip(id, train_number, origin, destination, departure_utc, arrival_utc, fare, capacity, enabled); Seat(id, trip_id, seat_number); Booking(id, pnr, user_id, trip_id, seat_id,

status, idempotency_key); PaymentEvent(id, booking_id, kind, status, reference); AuditEvent(id, actor_id, action, outcome, created_utc). Unique keys cover User.email, Booking.pnr, Trip+Seat number, and user+idempotency key. The API and database run in separate processes; database access is private to the API.

2.4 Security architecture

HTTPS terminates before the API. Auth service stores salted adaptive password hashes and enforces inactivity expiry and lockout. Every request is checked at the API and domain layer for role and booking ownership. Database constraints and parameterized queries reduce injection and race risks. AuditEvent records security-sensitive outcomes while omitting passwords and session tokens. Test data uses synthetic identities.

2.5 Architecture traceability

Auth service and API enforce FR-01, FR-02, SEC-01–04. Train search and database implement FR-03, FR-04, NFR-01. Booking service, payment simulator, and database implement FR-05–07, NFR-02–04. Admin service implements FR-08. AuditEvent implements FR-09. Web UI supports NFR-05–06.

3. Detailed design

3.1 Booking sequence

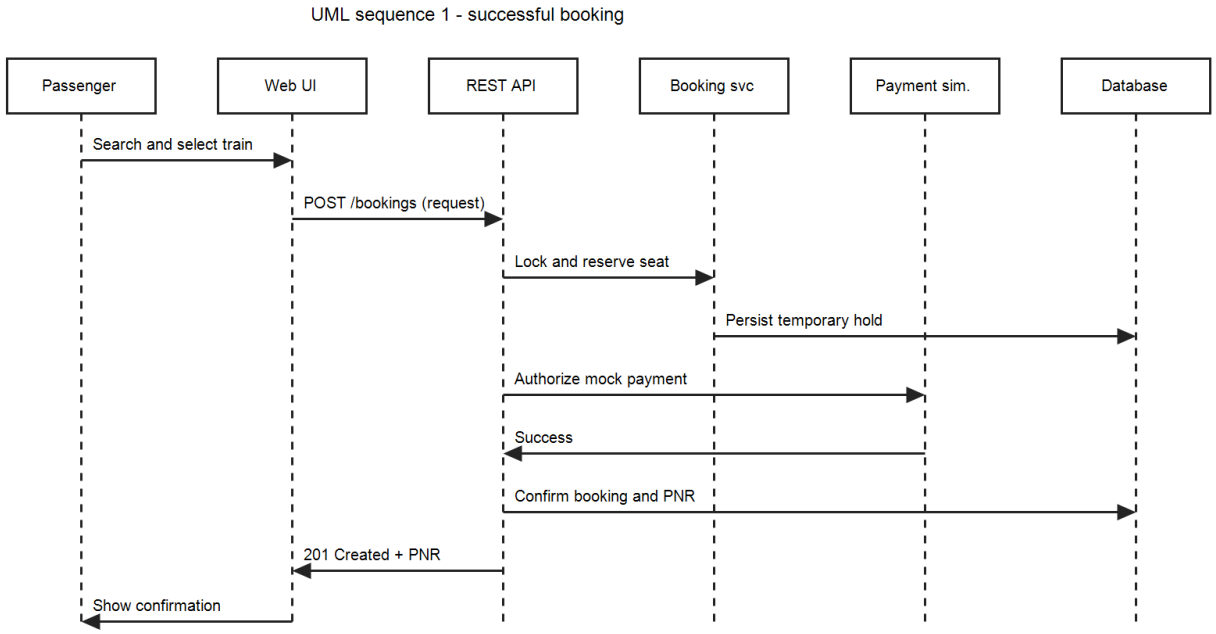


Figure 3. Successful booking sequence; failure releases the temporary hold without confirming a booking.

The booking service begins a transaction, selects an available seat with a database lock, and stores a short-lived hold. The payment simulator is called with a transaction reference. On success, the booking and PNR are committed; on failure, the hold is released. The idempotency key returns the existing result on retry. The seat uniqueness constraint is the final concurrency guard.

3.2 Cancellation sequence

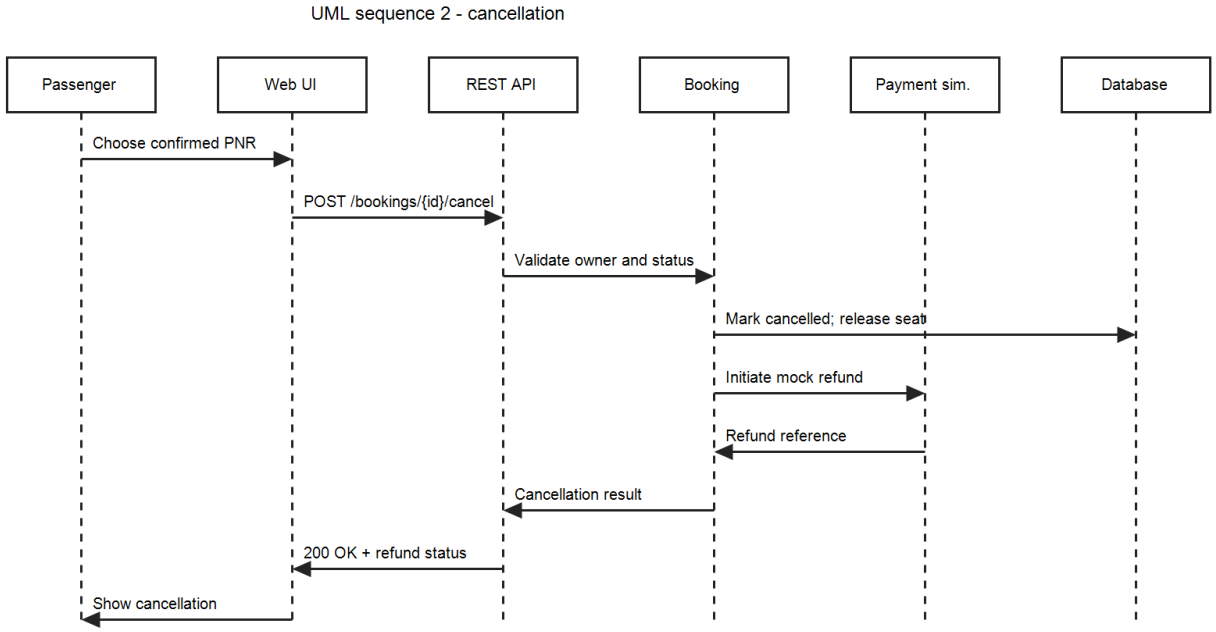


Figure 4. Cancellation sequence; the service checks owner and status before changing inventory.

Cancellation updates a confirmed booking to cancelled in one transaction and releases its seat once. A repeated request returns the existing cancellation result. The simulated refund reference is recorded with status, allowing a failed refund simulation to be visible without restoring the cancelled booking.

3.3 API design

POST /api/register → 201 or 409; POST /api/sessions → 200 or 401/429; DELETE /api/sessions/current → 204. GET /api/trips?origin=&destination=&date= → 200. GET /api/trips/{id}/availability → 200 or 404.

POST /api/bookings with tripId and Idempotency-Key → 201 with pnr, seat, status; 409 if unavailable; 402 for simulated payment failure. GET /api/bookings → 200 with caller-owned records. POST /api/bookings/{id}/cancel → 200 with cancellation and refund status; 403/404 for non-owner; 409 for ineligible booking.

POST /api/admin/trips → 201; PATCH /api/admin/trips/{id} → 200; DELETE /api/admin/trips/{id} → 204; GET /api/admin/audit → 200. All administrator routes require the administrator role. Request and response bodies are JSON; dates use ISO 8601 with UTC offset.

3.4 Error handling and state rules

Errors use a stable JSON shape: {"code":"...","message":"...","requestId":"..."}. Validation errors use 400, unauthenticated requests 401, forbidden requests 403, missing records 404, state conflicts 409, and unexpected failures 500. Error messages never include passwords, tokens, SQL, or internal stack traces. Transaction failure rolls back seat and booking changes. A booking moves from pending to confirmed or failed; confirmed moves to cancelled, never back to confirmed.

3.5 Design verification

TC-04, TC-05, and TC-08 validate the booking transaction and concurrency guard; TC-06 validates cancellation idempotency and ownership; TC-07 validates administrator separation and audit; TC-09 validates session and recovery behavior. Review the database constraints and endpoint contracts against their SRS IDs before implementation.

4. References and change control

Architecture reference: IEEE 1016:2009, https://standards.ieee.org/ieee/1016/4502/. Testing documentation reference: ISO/IEC/IEEE 29119-3:2021, https://standards.ieee.org/ieee/29119-3/7499/. Any change to an FR, NFR, or SEC ID requires a corresponding review of component mapping and test coverage.